

1 Write Amp

SSD erase units cover multiple pages. Outright in-place updating is expensive since erase granularity is coarse, so the SSD just marks pages as invalid instead. WA is the phenomenon whereas amt of physical space written is larger than the logical amount the application writes. Over-provisioning is the practice of setting the logical storage capacity lower than the physical capacity, effectively increasing the proportion of invalid pages. This makes it easier for the GC to find erase units with little live data left.

Best for all data in an erase unit to be rendered useless at the same time.

• Logical address space L

• Physical capacity P

• Usable capacity = $\frac{L}{P} = 1$

Pages in an SSD can be:

- Empty – writable
- Invalid – to be GC'd
- Valid – contains in-use data

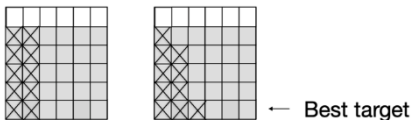
How do states relate to each other:

$$\frac{L}{P} = \frac{\text{valid}}{\text{empty} + \text{valid} + \text{invalid}}$$

1.1 WC Write Amp

Occurs: when invalid pages are uniform across erase units.

Worst case Non-Worst Case



Best erase unit to erase is the one with the most invalid pages.

SSD WA approximation: where x is % valid pages:

$$1 + \left(\frac{x}{1-x} \right)$$

WA Cost model assuming uniformly randomly distributed writes: where

$$\frac{L}{P} = \frac{\text{valid}}{\text{empty} + \text{valid} + \text{invalid}}$$

$$1 + \frac{1}{2} \cdot \frac{L/P}{1-L/P}$$

This gets multiplied by B if you are writing data that is too small.

1.2 Making the Most of your SSD

1. No small updates, updates take at least 4KB
2. Avoid random updates, since

$$GC_{WA} = 1 + \frac{\text{migrated}}{\text{freed}} = 1 + \frac{x}{1-x}$$

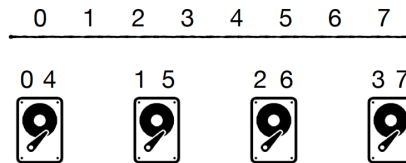
Basically random updates results in high garbage collection, since valid pages must be read from partially stale blocks

3. Prioritize R over W
4. R & W should be page aligned
5. Use async I/Os

2 RAID

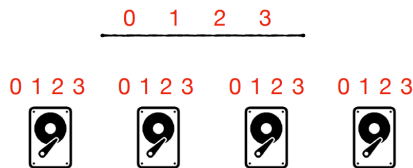
Virtualizes many drives at once to speed things up and provide fail-safes

2.1 Raid 0



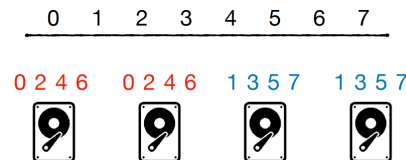
1. Seq R/W at combined bandwidth of all drives
2. Faster random reads/writes due to load balancing
3. No failure tolerance due to no redundancy

2.2 Raid 1



1. Writes as fast as bandwidth of single drive
2. Reads in combined bandwidth of all drives
3. Costs a lot of storage

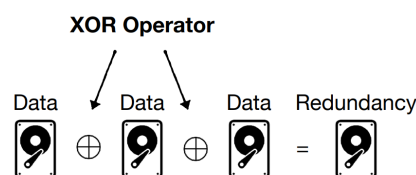
2.3 Raid 0+1



Stripe and mirror at the same time N drives, X drives per mirror group. Then:

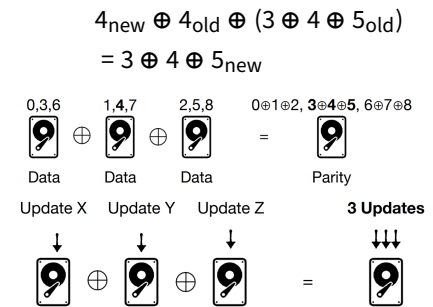
1. Sequential writes at $\frac{N}{X}$ bandwidth
2. Reads at combined bandwidth of N
3. Capacity: $\frac{1}{X}$ of total

2.4 Raid 4



Problems: performance bottleneck by the single drive of redundancy

Updating a whole stripe is easy, updating one page costs 2 reads and 2 writes



2.5 Raid 5



Raid 5 combines striping and distributed parity where redundancy is shared across all drives.

Random writes require 2 reads, 2 writes.

Sequential writes at $\frac{N-1}{N}$ of max bandwidth.

- Does not waste most of the storage capacity
- Random reads have less flexibility than with mirroring
- Fast sequential reads
- Random writes load is evenly distributed on all drives
- **Cannot handle more than one drive failing at a time**

3 Buffer Pools

The idea of buffers: keep hot pages in memory. Accessing the same pages over and over, given no writes in-between, are free

Why storing sequentially is better

Reading/writing from storage of units less than 4KB does not pay off:

- Disks: needles must move. For reading pages sequentially that stops becoming a problem
- SSDs: an entire page gets read or not at all

3.1 Tables

A table has:

- N entries
- B entries per DB page
- Whole table fits in $\frac{N}{B}$ pages

3.2 Continuous Scans

- Mixing table rows everywhere has WC $O(N)$ I/O reads

- Storing table rows contiguously has **WC** $O\left(\frac{N}{B}\right)$ I/O reads

3.3 Storing Pages

- Storing pages as a linked list, where previous pages point to the next requires synchronous I/Os, preventing SSD parallelism.
- Directories, each pointing to many singular pages does not saturate a disk's sequential bandwidth since each directory entry only points to one page
- Directories with extents (contiguous 8-64 pages) combine the best of both worlds and can grow into a tree

3.4 What each table stores

- Data type
- Size
- Start address

Databases keep track of free pages/extents by using a bitmap.

3.5 Supporting Indexless Deletes and Updates

Scan table, create holes for deletes and update in place for updates, $O(1)$ writes and $O\left(\frac{N}{B}\right)$ reads due to the need to read everything

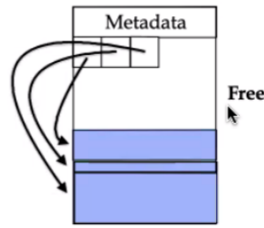
3.6 Optimizing Insertions

- Scanning and looking** for free space costs $O\left(\frac{N}{B}\right)$ reads and $O(1)$ writes.
- Storing extents and knowing which are full and which are not** reduces the read time to $O(1)$, cost of accessing directory table. Note: does not make variable-sized entry access faster
- Buffering insertions** eliminates the cost of reads and reduces write cost to $O\left(\frac{1}{B}\right)$ (every B writes perform a flush). Duplicates are allowed so no overwriting occurs.

3.7 Internal Page Organization

- Similar to a Python list (deletes take forever)
- Bitmap (no reorganization but requires more space)

3.7.1 Variable-Sized Row Organization



Using pointers instead of delimiters allow for random access, which is faster

4 Indexing Options

Non-B-Tree versions of indexing. This is where you might want to quickly know the page number given indexable item.

4.1 Zone Maps

Each page stores the min and max. No, this doesn't make it any faster – WC $O(N/B)$ I/Os

4.2 Sort The Column

Can only sort one column at once, search costs $O\left(\log_2\left(\frac{N}{B}\right)\right)$ I/O for a total CPU cost of $O\left(\log_2\left(\frac{N}{B}\right) + \log_2(B)\right)$, but updates cost $O\left(\frac{N}{B}\right)$.

4.3 Binary Tree

I/O and CPU cost for reads and writes are $O(\log_2(N))$. They don't exploit page locality.

4.4 Hash Table

Give up on sortedness and scan query optimization for average case $O(1)$. Map entries to page. Key repetitions are not allowed.

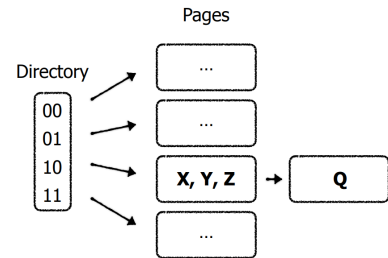
Hash table		Table	
hash(x) = x % 3		A	...
0	90 → 1	61	...
		7	...
		2	...
1	61 → 0 97 → 1	97	...
		90	...
		32	...
2	7 → 0 22 → 2	32	...
		74	...
		22	...

Downsides:

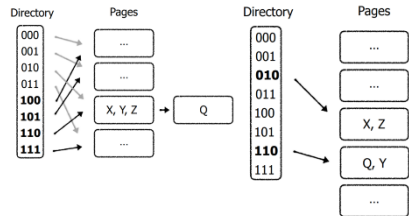
- Bad space efficiency after expansion (50%)
 - Only == comparisons
 - Expansion leads to performance slumps
- A fix for that?

4.5 Extendible Hashing

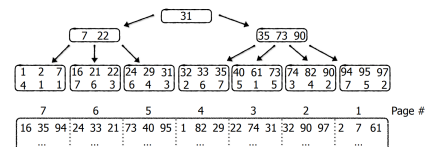
A directory maps pages in storage with a given hash suffix. Overflows are handled through chaining.



On capacity, double directory size. **New directory slots still point to previous pages.** Now only one overflowing bucket needs to be expanded at a time.

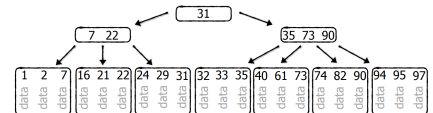


5 B-Tree



Where B is how many entries fit in a page:

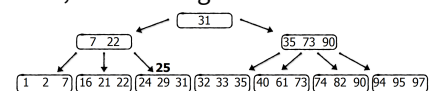
- Point search** costs $O(\log_B N)$, the height of the tree, since that's the pages it needs to search – $O(\log_2(N))$ CPU
- Writes** cost $O(\log_B N)$ reads and $O(1)$ writes. $O(1)$ reads if internal nodes are stored in memory, where $O\left(\frac{N}{B}\right)$ memory is used for that
- Scans** cost $O(\log_B N + S)$ reads. If the index is **clustered**, that is table data is stored within the index, that reduces to $O\left(\log_B(N) + \frac{S}{B}\right)$



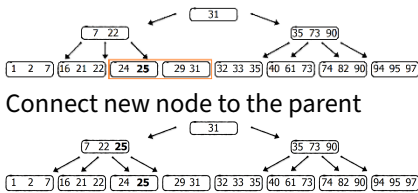
5.1 Going over Inserts (No Propagation)

How to insert?

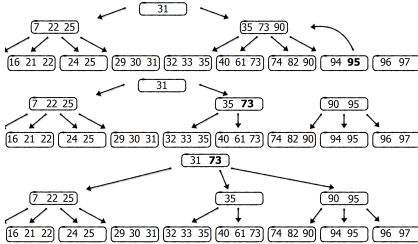
First, find the target node



If the node is full, split the target node



5.2 Inserts With Propagation



The read I/Os per insertion (assuming no storage in memory) remains $O(\log_B N)$, as we need to locate where to insert.

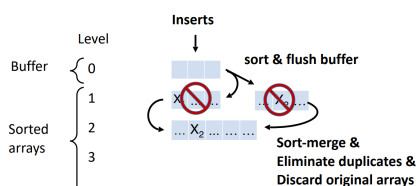
For writes:

- Every insertion updates 1 leaf
- One in $O(B^{-1})$ insertions triggers leaf split
- One in $O(B^{-2})$ insertions triggers parent split
- One in $O(B^{-3})$ insertions triggers grandparent split

5.3 Why Can We Just Store Internal Nodes In Memory?

Moreover, why do we usually assume that internal nodes are in the buffer pool? Because we access them so often. If a B-tree has $O(N)$ keys, $O(N/B)$ of them are stored in memory. In practice, this happens all of the time, and there are fewer internal nodes than there are keys in total.

6 LSM Trees



Stored as a B-tree, each sorted array is **Deletes**: insert a tombstone entry (the tombstone is a **value**).

Variables:

- P = no. entries that fit in the buffer (level 0). We assume LSM trees are clustered by default, so the size of an entry directly impacts P if memory remains constant
- Point searches cost

$$O\left(\underbrace{\log_2\left(\frac{N}{P}\right)}_L \cdot \underbrace{\log_B(N)}_{\text{B-tree search}}\right)$$

Since:

- $O(\log_2(\frac{N}{P}))$ levels to search
- $O(\log_B(N))$ cost of searching level (if it's organized in a B-tree) (only one sorted run per level)

6.1 LSM Tree Scans

Return most recent version of each entry in the range across the entire tree.

Scans cost:

$$O\left(\log_2\left(\frac{N}{P}\right) \log_B(N) + \frac{S}{B}\right)$$

From this point onwards, we will assume LSM trees store internal nodes for each B-tree in memory.

6.2 LSM Analysis Internal Nodes Memory

Queries (point)

$$\text{Searches} = O(L)$$

$$\text{Insertions} = O\left(\frac{L}{B}\right)$$

$$L = O\left(\log_2\left(\frac{N}{P}\right)\right)$$

Rationale: searches must go through each level. Insertions may result in a total of L write amplifications since each element can be merged a total of L times.

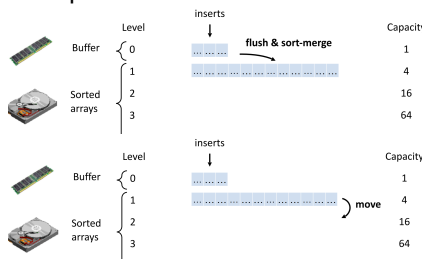
6.3 Levelled LSM Trees

Improves read performance at the expense of writes

Reduce number of levels by increasing size ratio T

Level count changes to $O(\log_T(\frac{N}{P}))$

Example: size ratio of 4:



Why does this slow down writes? Because elements get sort-merged more often. In the worst case, each element is re-written T times per level. If every element makes it to the bottom, inserts have a WA of $T \cdot L$.

Changes in cost:

$$\text{Lookup} = O(L)$$

$$\text{Insertion} = O\left(\frac{TL}{B}\right)$$

$$L = O\left(\log_T\left(\frac{N}{P}\right)\right)$$

Rationale:

- Less levels than the basic model
- Since only one sorted run per level, and **level count reduced**, looks up are faster
- Writing is more expensive by a factor of T

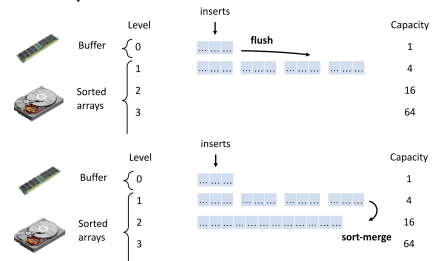
In the extreme case, where $T = \frac{N}{P}$ (or at most one level), **LSM tree becomes a sorted file**

6.4 Tiered LSM Trees

Improve write performance at the expense of reads

Reduce number of levels by increasing the size ratio T

Example: size ratio of 4



Why does this slow down reads? Because now I need to perform multiple searches per level, because there are more runs.

Changes in cost:

$$\text{Lookup} = O(TL)$$

$$\text{Insertion} = O\left(\frac{L}{B}\right)$$

$$L = O\left(\log_T\left(\frac{N}{P}\right)\right)$$

Increasing size ratio to $\frac{N}{P}$ turns the LSM tree into an append-only file

6.5 Conclusions I

LSM trees are:

- Write optimized
- Highly tunable
- Backbone of many modern systems
- Trade-off between lookup and insertion costs

6.6 Runtime Table

All internal nodes are stored in memory.

$$L = \log_T\left(\frac{N}{P}\right)$$

Basic: $T = 2$

\$	Apd. only	Sorted list	B-tree
Q	N/B	$\log\left(\frac{N}{B}\right)$	1
IR	0	N/B	1
IW	$1/B$	N/B	$1 + GC$
M	B		N/B

\$	LSM-BSC	LSM-TRD	LSM-LVL
Q	$\lg\left(\frac{N}{P}\right)$	LT	L
IR	$\frac{\lg\left(\frac{N}{P}\right)}{B}$	$\frac{L}{B}$	$\frac{LT}{B}$
IW	$\frac{\lg\left(\frac{N}{P}\right)}{B}$	$\frac{L}{B}$	$\frac{LT}{B}$
M	N/B	N/B	N/B

Notice how the write costs correspond to write amplification. The write costs correspond to the no. of times an element is copied in the worst case (if we factor out the B).

7 Bloom Filters

Bloom filters are probabilistic sets with the possible operations:

- Add
- Check for inclusion, where:
 - Negative returned for sure, or
 - Positive with error rate (FPR) of ϵ

It consists of k different hash functions, where its total size can be tweaked with a knob called “bits per entry”: M . With that, the total number of bits the filter takes up is $M \times N$. For that filter, check if an item is in there by running the k hash functions on the value you want to check. If one of the hashes correspond to 0 in the filter, it's a negative, else it's a positive.

7.1 Deciding no. of Hash Functions (K)

More memory $\Rightarrow$ lower FPR

No. of hash functions:

- 1 too low, FPR occurs every collision
- Adding more hash functions exponentially decreases the FPR
- Too many increases it again
- There's an optimal count depending on the bits per entry $M = \frac{m \text{ (total bits)}}{N}$

$$k^* = \ln(2)M$$

$$\Rightarrow \epsilon = 2^{-M \ln(2)}$$

7.2 Construction Contract

If we know:

- N entries to insert
 - ϵ = desired false positive rate (FPR)
- Then a filter should be allocated with $N \ln(2) \log_2\left(\frac{1}{\epsilon}\right)$ bits.

7.3 Bloom Filter Access Cost

$$\text{Insertions} = M \ln(2) = k = O(M)$$

$$\text{Positive query} = M \ln(2) = O(M)$$

$$\text{Avg. Negative query} = 2$$

$$\text{False Positive Rate} = 2^{-M \ln(2)}$$

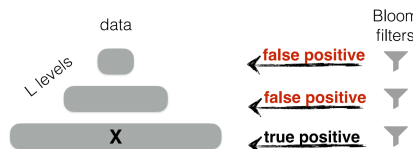
Rationale:

- Insertions: the number of hash functions that must run.
- Positive query: observed all the target bits were 1.
- Avg. negative query: observed one of the target bits was 0 and short circuited.

Note that “positive query” means querying something that's there, in this context.

7.4 Using Bloom Filters with LSM Trees

For a non-tiered LSM tree with L levels, each run of data needs a bloom filter. After all, it's meant to stop a potential I/O read to that level.



7.4.1 Filter Overhead for Basic Accesses

I want to perform a **point query**. The absolute worst case that could happen is **I search for an element that exists, it's on the bottom of the tree, and every bloom filter returns a positive**. Which means $L - 1$ false positives and 1 true positive, for a total **worst-case** cost of

$$\underbrace{(L - 1)M}_{\text{no. FPs}} + M =$$

$$\text{Worst case} = O(ML)$$

In the average case, false positives usually do not occur so the **average worst case** is:

$$2(L - 1) + M$$

$$\text{Avg. worst case} = O(M + L)$$

7.5 Optimizing Leveling

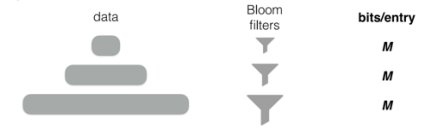
Let's optimize a leveled LSM. With bloom filters:

- Gets cost $O(2^{-M} \cdot L)$
 - Want $O(2^{-M})$
 - MONKEY stores this
- Inserts cost $O\left(\frac{TL}{B}\right)$
 - Want $O\left(\frac{T+L}{B}\right)$
 - DOSTOVESKY solves this

7.5.1 MONKEY (Get optimizer)

MONKEY OPTIMAL NAVIGABLE KEY-VALUE STORE

Bloom filters scale up the lower the level, **yet each bloom filter has the same FPR**.



If we're accessing our data on the bottom of the LSM, we want our bloom filters higher up to return true negatives, because the data higher is smaller, and we wouldn't want to waste a read.

Hence, we can relocate bloom filter sizes, **considering the bottom one is the largest**:



This shrinks the bloom filter on the lowest level and expands the bloom filters above it.

$$\left. \begin{array}{l} 2^M / T^2 \\ 2^M / T^1 \\ 2^M / T^0 \end{array} \right\} = O(2^{-M}) \quad \text{geometric progression}$$

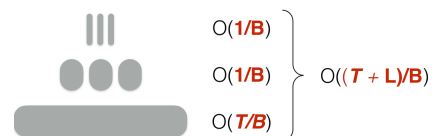
This results in a faster worst-case, $O(2^{-M})$ instead of $O(2^{-M} \cdot L)$. Of course this increases the absolute worst case up to $O(ML + L^2)$ but that never happens.

7.5.2 DOSTOVESKY (Write Optimizer)

DOSTOEVSKEY SPACE TIME OPTIMIZED EVOLVABLE SCALABLE KEY-VALUE STORE

We want to fix the cost of writes being proportional to the number of levels there are. Can be done by making writes lazy on the upper levels. Therefore:

- Apply tiering on the upper levels
 - Apply leveling on the lowest level
- This reduces the insert I/O cost from $O\left(\frac{TL}{B}\right)$ to $O\left(\frac{T+L}{B}\right)$.



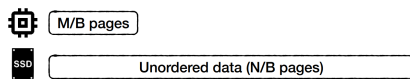
8 External Sorting

Mergesort is worst $O(n \log(n))$ and takes $2x$ more space. Quicksort has average worst case $O(n \log(n))$ and is done in-place. These only work in memory. So, **what is the optimal way to sort lots of data in storage?**

Here are some attempts:

- B-tree
 - Scan unordered data, insert each into B-tree, append afterwards for $O(N \log_B N)$ R and $O(N)$ W
 - If internal nodes in mem: $O(N)$ R and W
- LSM-tree
 - $O\left(\frac{N}{B} \log_2\left(\frac{N}{P}\right)\right)$ reads and writes all the time

How can we efficiently sort data that doesn't fit in memory?



Setting up variables:

- M entries fit in memory
- N entries in total
- B entries per 4KB page
- $\Rightarrow M/B$ pages in mem, N/B pages to process in total

$$\begin{aligned} \text{Iterations} &= \log_{M/B} N/M \\ &= \log_{\text{pages in partitions}} \text{memory} \\ &= \log_{P_{G_{\text{mem}}}} \text{Partitions} \end{aligned}$$

Each iteration corresponds to how many levels of merging need to be done.

Each iteration does a full pass over data for a cost of $O(N/B)$

Merging costs $\frac{N}{B} \left[\log_{\frac{M}{B}} \frac{N}{M} \right]$ alongside N/B for partitioning.

Total calculated by

$$\underbrace{\frac{N}{B}}_{\text{partitioning}} + \underbrace{\frac{N}{B} \log_{\frac{M}{B}} \frac{N}{M}}_{\text{merging}} =$$

$$\text{Total } O\left(\frac{N}{B} \log_{\frac{M}{B}} \frac{N}{B}\right).$$

Useful quantities:

- $\frac{N}{M}$: no. partitions
- $\frac{N}{B}$: total page count
- $\frac{M}{B}$: pages in memory
- M : entries that fit in memory
- N : entries in storage

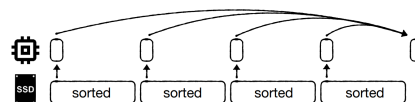
Increasing memory count means merging more partitions per pass. How much memory is required to merge **all** partitions in one pass? (I mean 2) – solve for M :

$$\begin{aligned} \log_{\frac{M}{B}} \frac{N}{B} &= 2 \\ \Rightarrow M &= \sqrt{N \cdot B} \end{aligned}$$

With this M , memory can accommodate $\sqrt{N/B}$ buffers.

With $M = \sqrt{N \cdot B}$ memory to partition data, this creates at most $N/M = \sqrt{N/B}$ sorted partitions. Then merging in one pass uses at most $\sqrt{N/B}$ input buffers.

For the cost of $O(N/B)$ I/Os overall. The cost of this:



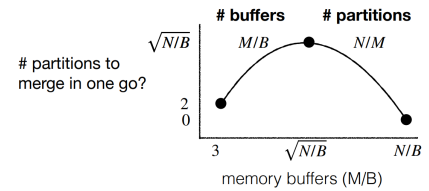
For all practical purposes, a 2-pass sorting-algorithm is practical.

8.2 Optimizing CPU cost

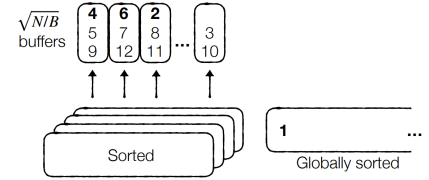
We expect $O(N \log_2(N))$.

Partitioning and sorting each chunk for a total of N/M chunks cost $O(N \log_2(M))$.

For merging:



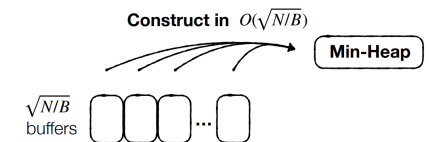
The idea is to traverse all $\frac{M}{B}$ buffers (e.g. $\sqrt{N/B}$ at the optimal) and pick minimum:



For a total of $O(\sqrt{N/B} \cdot N)$ comparisons.

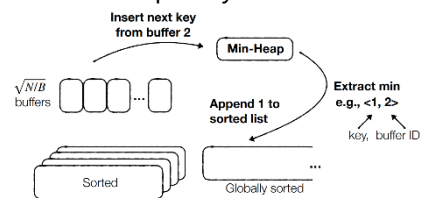
How do we sort this more efficiently?

For a min-heap with $\sqrt{N/B}$ items, one per buffer, we're able to **pop then insert** in log time.



So, for all buffers, the first elements of each buffer go in the min-heap. Each heap entry stores the buffer ID.

When item is extracted, insert next key from the heap entry's buffer ID.



Per entry, this costs $O(\log_2 \sqrt{N/B})$. For N items that must be processed, this costs $O(N \log_2(N/B))$.

In all:

- **Partitioning** costs $O(N \log_2(M)) = O(N \lg(\sqrt{NB}))$
- **Merging** costs $O(N \lg(\sqrt{N/B}))$
- Total cost is $O(N \log_2 N)$

The same as in-memory algorithms. Thus, the overall cost is:

$$\begin{aligned} O(N \log_2(N)) & \text{ CPU} \\ \left\{ \begin{aligned} O\left(\frac{N}{B} \log_{\frac{M}{B}}\left(\frac{N}{B}\right)\right) & \text{ I/O} \\ O\left(\frac{N}{B}\right) & M > \sqrt{MN} \end{aligned} \right. \end{aligned}$$

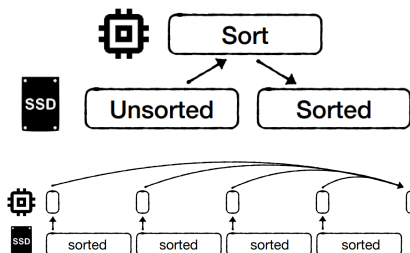


8.3 Double-Buffering

Doubles the space required, 2 buffers required per partition, but eliminates the

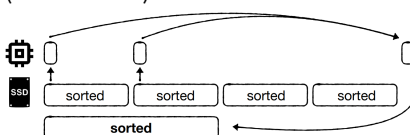
8.1 Multi-Way Merge Sort

For each chunk that fits into memory, sort them for N/M sorted temp. files for a cost of N/B I/Os (that is the partitioning phase)



Allocate a buffer for each input file and merge into output stream.

Too little memory requires multiple passes (here: 2 iters).



There are $\frac{N}{M}$ partitions, and we have $\frac{M}{B}$ buffers (pages in memory). **Each insertion merges $\frac{M}{B}$ partitions.** Total iterations:

need to wait when an individual buffer runs out.

9 SQL Query Optimization

Firstly, statement trees should be the most furthest from balanced.

9.1 Selection

The index uses a B-tree.

For the simple `SELECT * FROM ... WHERE A = ...`

- Scans are just $O\left(\frac{N}{B}\right)$
 - Unclustered indexes are $O(\log_B(N) + S)$
 - Clustered indexes are $O(\log_B N + \frac{S}{B})$
- (Same table) When there are multiple selection conditions, like `SELECT * FROM ... WHERE X = "i" AND Y = "j"`
- Either unclustered (iterate cluster, check row)
 - Cost $\log_B(N) + |X_i|$ or $\log_B(N) + |Y_j|$, depending on which cluster is checked (ideally, should check the one with the smaller $|X_i|$ or $|Y_j|$)
 - Both unclustered
 - $2 \log_B(N) + \frac{|X_i|}{B} + \frac{|Y_j|}{B} + |X_i \cap Y_j|$
 - Composite index
 - $\log_B(N) + |X_i \cap Y_j|$

When taking the "either unclustered" approach, index through the table with less items.

Searching both indexes is better when $|X_i \cup Y_j|$ is small relative to $|X_i|$ and $|Y_j|$.

9.2 Order By

Any time there's an ORDER BY statement: unclustered index is helpful if result set is small, while a clustered index scan is good in all cases.

Quicksort, if result set fits in memory.

Heapsort if quicksort takes longer than expected, since heapsort has true $O(1)$ aux. space complexity.

9.3 Distinct

Sort and eliminate identical items, OR hash to identify identical items.

Sorting is better if you are going to need to sort the data anyway.

9.4 Group By

Index/sort by category, or hash by category.

9.5 Joining

Methods:

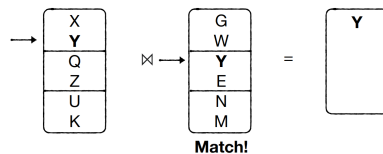
- Nested loop

- Cost $\frac{|T_1||T_2|}{B}$
- Strawman algorithm "slow"
- Block nested loop
 - Cost $\frac{|T_1||T_2|}{B^2}$
 - If one table fits into memory: $\frac{|T_1|}{B} + \frac{|T_2|}{B}$
 - If we read Q pages at once from T_1 for each scan of T_2 : $\frac{|T_1|}{BQ} \cdot \frac{|T_2|}{B}$
- Index join
 - Use when indexes on the join keys
 - Either w/ T_2 indexed: $\frac{|T_1|}{B} + |T_1| \log_B |T_2|$
 - Both: $\frac{|T_1|}{B} + \frac{|T_2|}{B} + |T_1| + |T_2|$
 - Same for clustered/unclustered
- Sort-merge join
 - Use when both relations much larger than memory
 - I/O cost $\frac{|T_1|}{B} + \frac{|T_2|}{B}$
 - CPU cost $|T_1| \lg |T_2| + |T_2| \lg |T_1|$
 - Mem cost $\sqrt{\max(|T_1|, |T_2|) B}$
- Grace hash join
 - I/O cost $\frac{|T_1|}{B} + \frac{|T_2|}{B}$
 - CPU cost $|T_1| + |T_2|$
 - Memory cost $\sqrt{\min\left(\frac{|T_1|}{|T_2|}, \frac{|T_2|}{|T_1|}\right) B}$

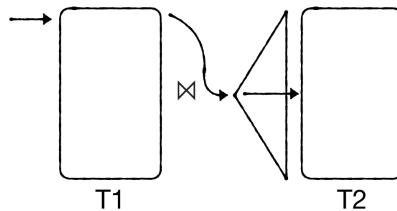
How block-nested loop works:

Block Nested Loop

For each block in one relation, scan whole other relation

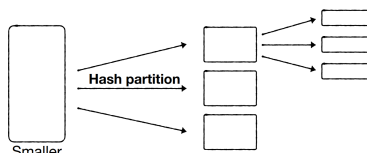


How index join works



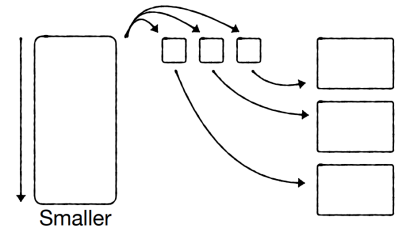
How grace hash join works (best for very large relations that do not fit in memory):

(A) Hash partition smaller table
Repartition until each partition fits in memory

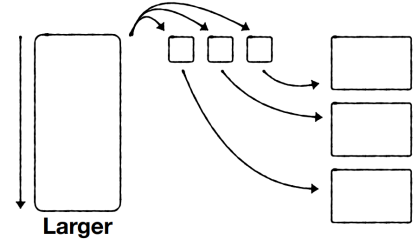


This requires one pass and multiple buffers. **Must repartition smaller table until each one fits into memory.**

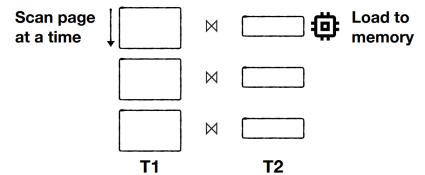
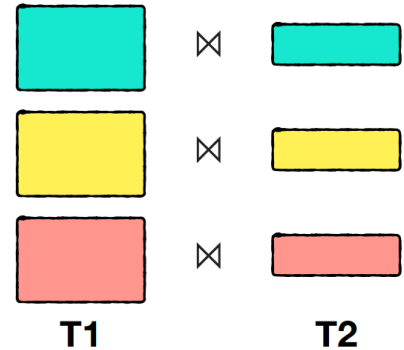
Requires one pass and multiple buffers



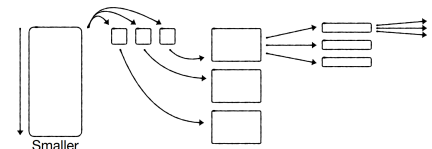
For the larger table, hash partition it again using the same hash function. This time, each partition can be larger than memory.



Afterwards, we join each pair of matching partitions independently. Only a pair of matching partitions can have joining keys since we hash partitioned.



When partitioning smaller table, number of available buffers dictates how many iterations we need until all partitions are in memory.



Assuming M entries fit in memory, the no. of iterations required is:

$$\log_{\frac{M}{B}} \left(\frac{|Smaller|}{M} \right) + 1$$

Where $\frac{M}{B}$ is the partitioning fanout, and $\frac{|Smaller|}{M}$ is the no. of partitions we must divide table into such that each fits into

memory. Equating this to 2 and solving for M gives us the memory we need to join in two passes. **If we have lots of more memory**, we can join to in-memory partitions immediately, reducing the pass count from 2 to 1.

As memory increases, this becomes identical to block-nested join.

9.6 Cardinality Estimation

Estimate X_i as $\frac{N}{|X|} = \frac{\text{no. rows}}{\text{unique } X \text{ values}}$

The problem: we don't know N , since on insertion we do not know if the value is unique unless we have an index

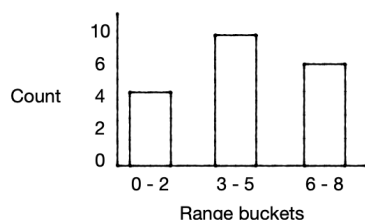
You can put a cardinality counter for each index

If no index:

Periodically scan and count no. of unique entries

We can instead use histograms, with range buckets. Estimate $|X_i|$ as $\frac{\text{bucket count}}{\text{bucket range}}$

Histograms



9.7 Query Optimization

Projections should drop unneeded columns.

- **Reorder Boolean predicates by selectivity.** When evaluating a WHERE clause, evaluating the **most selective** predicates first is the most helpful.
 - i.e. if age > 60 (selects 10%), country = US (selects 50%), status = active (selects 80%), then run the selection from left to right.
- **Reorder join trees** so that the smallest intermediate result sets appear first.
 - i.e. if $A \bowtie B$ returns 1% of a and 20% of B , but $B \bowtie C$ returns 70% of C , then the join operation should be $(A \bowtie B) \bowtie C$
- **Drop completely impossible queries.**
- **Push projections** as deep as possible in the AST (so that they get evaluated first) so that every operator only produces the columns that the parent actually needs. At each AST node, determine which attributes are required higher-up. Discard the rest. (You may σ first before π)

10 Column Stores

Great for large statistical calculations and batch updates. Updating one at a time is way too expensive. Also, items should still be fixed length.

Scanning a column in memory would require hashing overheads for each page. Therefore, just read and map at column granularity.

Basic model: a store contains $A[]$, $B[]$, $C[]$, and so on. Different columns are not contiguous to each other.

10.1 Processing Queries

In a column store: `SELECT min(C) FROM Table WHERE A>10 AND B<20`

Approaches:

- **Early materialization (the dumb way):** For each item: Check A – if pass, then B – if pass then deal with C
 - Unfortunately, more random I/Os and more CPU branch mispredictions, and incompatible with SIMD
- **Late materialization: process one column at a time**
 - **Scan column A, fetch qualifying IDs, and then start scanning B**
 - Works best when A has the fewest qualifiers (best to filter on more selective columns first)
 - Requires more memory to withhold intermediate results, **but entails better cache behavior and CPU efficiency**
 - For intermediate results:
 - * **Array of integer IDs work best if few results qualify**
 - * **Bitmap good when many results qualify**

Ways to optimize:

- Use zone maps (each partition stores min and max)
 - Store columns sequentially to allow pre-fetching and I/O parallelism
- To make filtering more efficient, filter on more selective columns first to reduce size of intermediate results and thus I/O**

10.2 Handling Insertions

Operation: `INSERT into TABLE VALUES (a, b, c)`, approaches:

- **In-place:** directly insert to the end of each column by reading/writing to storage. Cost: $O(\text{no. cols})$
- **In-memory buffering:** I think you get the idea

10.3 Handling Deletes

There couldn't be a better way.

- Bad idea: Create holes.
 - Now you need an additional bit per column to track if an entry has been deleted
- Employ delete column
 - An additional column in the table signals that an entry has been deleted. **To make updates work, we store the deletion timestamp for each delete operation and store the insertion timestamp for each insert.**

10.4 Handling Updates

For query `UPDATE TABLE SET B="b1", C="c1" WHERE A="x"`

Delete and re-insert. Store the deletion timestamp for each delete operation and store the insertion timestamp for each insert.

There is no better way. This is why modifications are best done in batch and offline.

10.5 SIMD

SIMD operations can apply one instruction in parallel to multiple values within one cache line.

Select sum(A) from table where A > 5



3	1	7	4	8	9	2	8
>	>	>	>	>	>	>	>
5	5	5	5	5	5	5	5

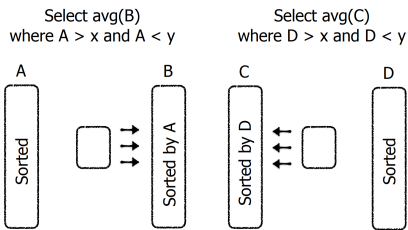
10.6 Compression

Options:

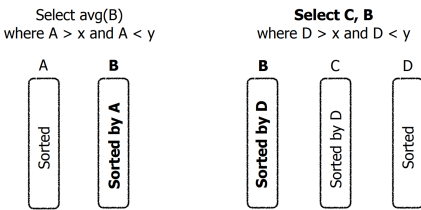
- Bit-vector encoding: Employ one bit string for each possible value indicating if the entry has the given value (this effectively makes each entry an Enum)
- Dictionary encoding: Employ a dictionary with smaller strings to represent larger ones. Slower as we need to read more bits
- Run-length encoding
 - Cat, Dog, Cat*4, Horse, Cat*2, Dog
 - Compatible with dictionary encoding
 - Lose out on index access

10.7 Indexing In Column Stores

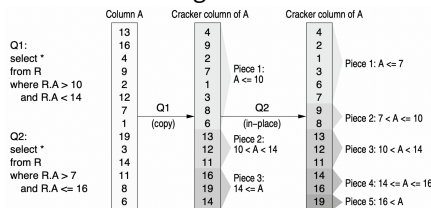
It's still useful to index columns to make it easy to find where something is. However, that still isn't good enough. Our best approach is to **sort the columns**.



Unfortunately, this might result in columns being duplicated.



We solve this problem by employing database cracking:



For Q1: partitions the table into $A \leq 10$, $10 < A < 14$, $14 \leq A$. Cracking up to what's necessary. Future operations can crack harder.

- Queries adaptively refine the sorting and speed up subsequent queries
- We can begin querying the data immediately without creating too many projections from the onset, which would take a while

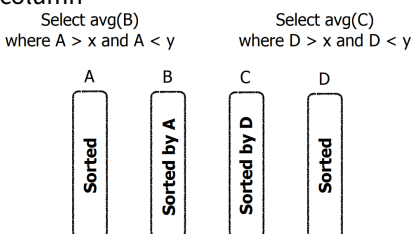
10.8 Column Groupings

Physical layout optimization for columns frequently accessed together. Group the columns together so that a single entry in a column contains multiple data.

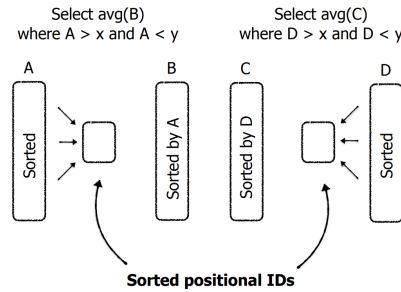
10.9 Column Projections

A projection groups certain columns together into a materialized structure so queries that frequently access those columns can read a minimal amount of data.

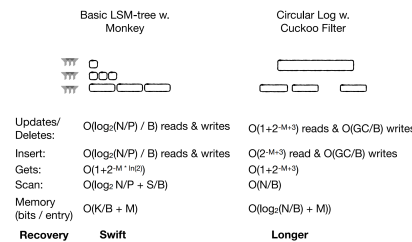
Example: sort subset of columns by one column



Like having multiple clustered indexes on a subset of columns



11 Circular Logs, Cuckoo Filters

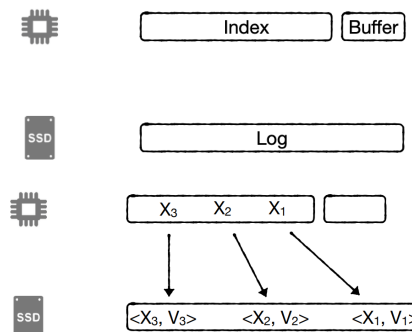


Circular logs have the cheapest writes, takes the most memory, but does not support scans. These are the ideas:

- Mechanics
- Hot/Code separation
- Checkpointing and recovery
- Cuckoo filtering

For now, treat this as a KV store.

11.1 How it Works



- Buffer stores KV pairs
- Index stores K → Block ID in the log pairs
- Log stores KV pairs (same as buffer)
 - When buffer full, flush to the end of log (treating this as a write-only log)

If only insertions:

- Writes cost $O\left(\frac{1}{B}\right)$
- Gets cost $O(1)$

11.2 Supporting Deletes

Deletes: remove entry from buffer
Requires garbage collection if there are too many orphaned entries.
Divide log into equally-sized areas (can be in the order of GBs)
Memory now stores a counter of the number of bytes that are stored per block

When deleting:

- Get the entry in the log to check its size
 - Subtract size from area counter
 - Remove entry from index
- OR to avoid this, we store the size in the buffer

11.3 Updates

Delete then re-insert

11.4 GC

Pick area with least live data left. Scan area for each entry.

If key is not indexed in memory, the entry must have been deleted, so don't carry it on.

If the key is alive (indexed) but pointing elsewhere, then don't carry on

If the key is indexed and pointing to the entry, then migrate it

11.4.1 When To Trigger

When this value: $\frac{\text{Live Data}}{\text{Physical space}}$ reaches a threshold

How much WA? x = percentage of valid pages in areas that are to be garbage collected

WA: Mixing hot/cold data brings us to the worst case. **If we do not separate hot and cold data, this will be the case**

$$1 + \left(\frac{x}{1-x} \right)$$

11.5 Hot-Cold Separation

We do not want to garbage-collect in blocks that store cold data. So, we have this:

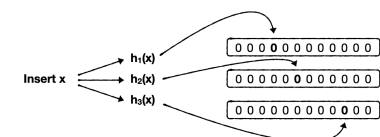
- User updates are hot
- Garbage-collected data is cold

So separate them.

Better solution? Separate data with different temperatures to different areas
How do we tell the temperature of the data? Without taking that much space?
(Store the number of access to that data), use a Count-Min

11.6 Count-Min

Data structure that reports the frequency of elements in a data stream



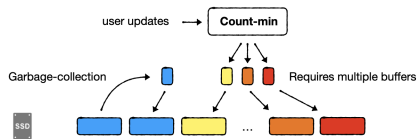
Insert an entry by hashing it to one counter in each array using all the hash functions, and increment that counter

To query for the frequency of a key, hash it to each array and return minimum
 This result is a guaranteed upper bound of the real count
 Result might overestimate the true answer due to hash collisions
 $\text{Estimated freq} \leq \text{True frequency} + \epsilon \cdot \text{num insertions}$
 Where ϵ is a parameter, with probability $1 - \delta$

$$w = \left\lceil \frac{e}{\epsilon} \right\rceil$$

$$d = \left\lceil \ln \left(\frac{1}{\delta} \right) \right\rceil$$

These parameters define: the no. counters (width) and the no. arrays (height)



11.6.1 Cooling Down

Every x insertions, we can divide all counters by 2
 In addition, we can not add to values at max

11.6.2 Const Analysis

Write cost: $O\left(\frac{GC}{B}\right) I/O$
 Read: 1

11.7 Recovery

If power fails, we lose: index, live bytes, buffer
 It is okay to lose the buffer
 Recovery algorithms:

- Scan from newest to oldest, and add the entries back to the index (knowing not to add duplicates once encountered), doable in one pass

Or we can use checkpointing:

- Every X updates or insertions, store a copy of index and live bytes
- After recovery, load copy then scan X latest updates / insertions to check for invalid pointers (if something may have been updated or deleted) and remember to update Live Bytes as well

Cost: additional write cost $O\left(\frac{\text{index size}}{X}\right)$,
 recovery time: $O(X/B)$ reads for a backwards scan

So total cost:

Write: $O\left(\frac{GC}{B} + \frac{\text{index size}}{X}\right)$

Recovery time: $O\left(\frac{X}{B}\right)$

Get: $O(1)$

Note that deletes can be lost. It is better to just store tombstone entries.

11.8 Cuckoo Filters

Index sizes can be large. It is:

$$\frac{N(P+K)}{\alpha}$$

Where:

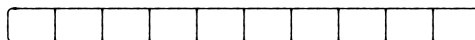
- N is the data size
- P is the pointer size = $O\left(\log_2 \frac{N}{B}\right)$
- K is the key size = $O(\log_2 N)$
- α is the collision resolution overheads, less than 1 = 0.8 by default

We can attack the key size issue with cuckoo filters.

And we can tackle α with cuckoo hashing to avoid hashing collisions.

11.9 Cuckoo hashing

Table of buckets, each containing 1 entry.
 Two hash functions.



Insertions: insert x at $h_1(x)$

If z is already in $h_1(x)$, then move z to $h_{\text{other}}(z)$ (using the alternative hash function) -

“other” is the hash function that was not used last time. Continue doing this recursively until all entries are mapped to an empty bucket. That’s where the word cuckoo comes from. It just evicts.

In this case, a query has to check at most two locations to find a key.

Insertions may endure multiple evictions and swapping of keys between buckets. Also, an infinite loop is possible, but you may limit the loop count to N .

```
def insert(x):
    if T[h1(x)] == x or T[h2(x)] == x:
        return # already
    pos = h1(x)
    for _ in range(n):
        if T[pos] is None:
            T[pos] = x
            return
        # swap
        x, T[pos] = T[pos], x
        if pos = h1(x):
            pos = h2(x)
        else:
            pos = h1(x)
    rehash()
    insert(x)
```

We can alleviate that by allowing multiple keys per bucket.

Insertions succeed in $O(1)$ with high probability

- Assuming load factor <50% for bucket size 1
- Assuming load factor <84% for bucket size 2
- Assuming load factor <95% for bucket size 4

So, this type of hashing is great at reducing collision overheads, /0.8 to /0.95

11.10 Cuckoo Filters

Attacking the key size problem because keys are too large, and they need to be larger the more data there is. Keys can be arbitrary large and are variable length. So instead:

- We store fingerprints instead of keys in the buffer



A fingerprint is a hash digest derived by hashing a key:

$FP(\text{key}) = \text{fingerprint}$

We store $FP(X)$ in $h_1(X)$

And if we need to evict something, move existing element in $h_1(X)$ to $h_2(X)$

We should be able to derive $h_1(X)$ from $h_2(X)$ and $FP(X)$ and the other way around, because we want such mappings to be reversible.

We do this by XOR-ing.

So, let’s XOR the bucket address and a hash of the fingerprint. And then we define:

$$h_2(X) = h_1(X) \oplus \text{hash}(FP(X))$$

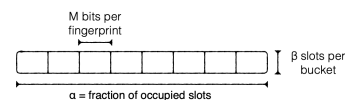
The result mapping is reversible.

When querying, query $h_1(X)$ and $h_2(X)$. If we find a matching fingerprint, we report a positive. However, there can be false positives. In the case $FP(Y) = FP(X)$

No, false negatives are not possible.

The FPR is $2 \cdot 2^{-M} \cdot \beta \cdot \alpha$

$$\text{false positive rate} \approx 2 \cdot 2^{-M} \cdot \beta \cdot \alpha$$



When at capacity, $\alpha = 0.95$

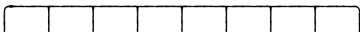
Hence FPR is actually

$$2^{-M+3}$$

Better than BF for $M \geq 10$, where the FPR for BF is $2^{-M \ln(2)}$

For memory accesses:

	Cuckoo Filter	Bloom filter
false positive rate:	$\approx 2^{-M+3}$	$2^{-M \cdot \ln(2)}$
Memory accesses for positive query	1.5	$M \cdot \ln(2)$



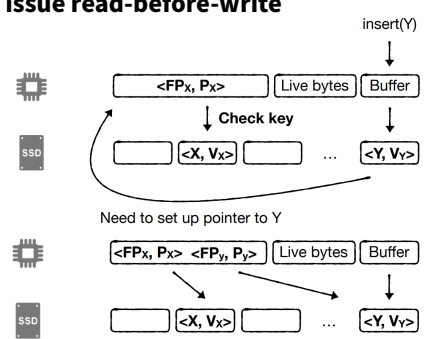
For negative queries, it's 2 memory accesses
Insertion cost?

- Cuckoo is expected $O(1)$
- For BF, it's $M \ln(2)$

11.11 Read-Before-Write Necessity

Cuckoo filters store fingerprints (hashes) rather than full keys. Without looking at storage, you will not know if that matching fingerprint corresponds to the key you're inserting to, or a key you aren't. In other words, **insertions could not be told apart from updates.**

Suppose we insert key Y that has a matching fingerprint to existing key X . **To safeguard against orphaning, we must issue read-before-write**



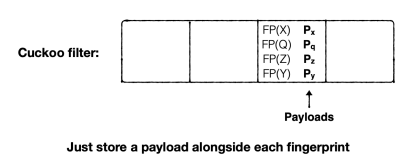
If a read-before-write was not assumed, $\langle FP_x, P_x \rangle$ would be re-routed and $\langle X, V_x \rangle$ would be orphaned. We do not want that.

11.11.1 Cuckoo Filter Deletes

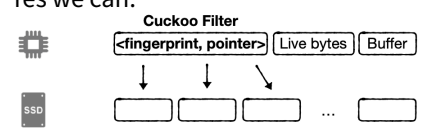
A delete searches both buckets for a key and removes matching fingerprints. **We do not want to delete fingerprints for a key that was never inserted. We must check if the entry in the filter corresponds to the entry in storage we want to delete.**

As long as we delete keys we know for sure that have been inserted, no false negatives can occur, which works if we have matching fingerprints for different keys in the same bucket.

11.12 Storing Payloads in a Filter



Yes we can.



12 Transaction Management

A database must maintain consistency. There is no intrinsic notion of a consistent state. This is defined by the user. To maintain consistency, database systems give the notion of transactions. It is a **series of operations on DB** from one consistent state to another.

Consistent state $\rightarrow$
Transaction $\rightarrow$ Consistent state

For example, in a bank transaction:

- $A = A - 100$
- $B = B + 100$
- Both must succeed or fail otherwise inconsistent state

Transactions do not need to interact with each other. A transaction is oblivious to everything else.

12.1 Transaction API

- Begin transaction
 - Some transactions
 - Commit or abort
- For example:
- $b = \text{select balance from accounts where id} = x$
 - If $b \leq 100$
 - Abort
 - Else
 - Update accounts set balance $= b - 100$ where $\text{id} = x$
 - Commit

12.2 Semantics for Transactions

ACID means:

- Atomicity
 - All or nothing
- Consistency
 - Transition across consistent states
 - Only the user is responsible for this
- Isolation

- Not corrupted by concurrency
 - Durability
 - Recover from failure
- The user is responsible for defining what a consistent state is. They define the integrity state and what the transactions are. The database is responsible for everything else.

12.3 What Can Go Wrong?

- System Failure
- Concurrency

12.3.1 System Failure

On a power failure:

- $A = A - 100$
- Power fails
 - Power fails before the transaction finishes, leaving the system in an inconsistent state
- $B = B + 100$

What should happen?

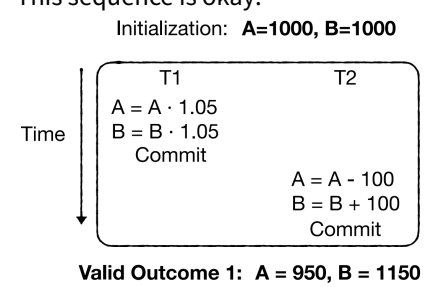
If system goes back online, we make sure we cancel out the impact of the first operation.

12.3.2 Concurrency - Dirty Read

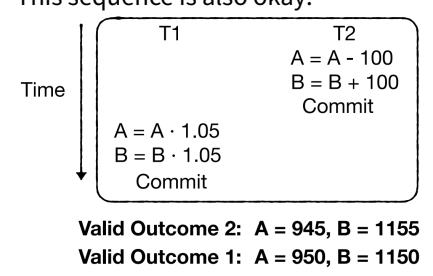
Concurrent transactions can result in an inconsistent state. Examples:

Interests and payments

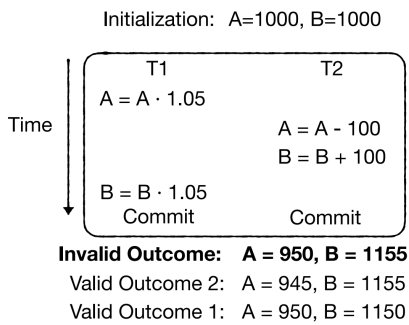
This sequence is okay:



This sequence is also okay:



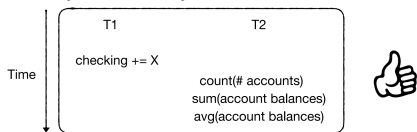
But this sequence is not:



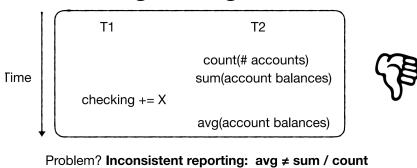
The anomaly is a **dirty read**. T2 reads a modified but uncommitted data item from T1.

Updates and statistics – Unrepeatable Read

Example: for a specific user



What could go wrong?



Operations defined in a transaction should return consistent results. If Balance += X occurs in the middle of the statistical operations, it results in inconsistent reporting: $avg \neq \frac{sum}{count}$. This is an **unrepeatable read anomaly**: where subsequent reads of the same data are inconsistent as the data is changed in-between.

12.4 Solutions

The simplest solution of dealing with all of these concurrency problems is to just lock the database for any modification you want to make. The problem? It's slow (terrible for performance).

In particular: DBMS should really saturate all the hardware that is available. For example, if one transaction is waiting for an I/O to complete, you don't want to wait until that completes. You want to use other resources.

Also, a short transaction should not have to wait until a long transaction completes (convoy effect).

A good compromise: serializability.

This means that transactions in a database can be concurrent, as long as they actually result in a consistent state. A consistent state is where it should have been achievable through a serial execution of the transactions (any order is fine). In other words, the final outcome is equal to **some**

serial execution of the transactions.

So, how do we achieve concurrency and serializability at the same time? We want some sort of locking protocol, a generic one that allows this to be achieved.

12.5 Strict Two-Phase Locking

This is a two-phase commit, where a transaction is divided into two phases.

Phase 1: lock every data item as encountered for the first time. Reads take shared read locks to prevent writes, writes take exclusive write locks (i.e. if something is changed, no one should read). **If the read lock is taken, data cannot be modified, i.e. write lock cannot be taken. If the write lock is taken, no other locks can be.**

Phase 2: release all locks during commit.

What invariant does this guarantee: a data item that has been accessed by this transaction cannot be modified by another transaction until this transaction commits. How does this allow us to fix interest payments and unrepeatable reads?

- Write Lock A
- A × = 1.05
- Write Lock B
- B × = 1.05
- Commit
- (Repeat with + instead of ×)

This uses exclusive write locks

Another example:

- Apply a read lock all data
- Compute avg
- Compute sum
- Compute count
- Commit
- Update data
- Commit

This uses share read locks

12.6 Implementing Locks

A database system has a lock manager. It is a hash table the database maintains that hashes from the ID of the object being locked, to some information of the lock. Object ID → (type, lock count, queue of waiting requests)

Where:

- Type: shared or exclusive
- Lock count: how many transactions are using this shared lock
- Queue: Transactions waiting on an exclusive lock (what to invoke next when this lock is released)

You can lock a row, page, or a table. If you have a very selective query, it may make sense to lock certain pages or rows. If you

don't have a selective query, it might make sense to lock the entire table.

We lock an object the first time it is accessed by a transaction. Locking table implemented by OS locking primitives (using mutexes)

12.7 Latches

Locks separate transactions, latches separate threads.

Locks protect DB content, latches protect in-memory structures such as an LRU queue.

Where lock lasts for a transaction, where a latch only lasts for the duration of a critical section.

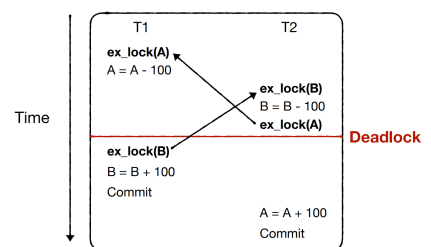
Locks use a lock manager, latches use spin-locks.

12.8 Updating Indexes

Where Update table set C where A

1. Begin transaction
2. Search index A
3. Access row and update C
4. Update entry in C's index
5. Commit

12.9 Resolving Deadlocks



For concurrent payments, there is a risk of deadlock.

What should we do when we detect a deadlock? And no, locking the entire thing is wasteful.

When we have a deadlock, we need to stop and abort all transactions. This means that we have to undo. This is done by capturing the state of the data item before it is modified.

We record the before-image of all changes to the DB in a sequential log. How do we actually do this? We use a transaction log.

- T1 starting
- A: 1000 → 900
- T1 commit

In the sequential log, we are recording the before-state of all the items as we modify them. Why is that good? As soon as we have an abort, in the sequential log, we can traverse it backwards and use them to undo the changes.

12.10 Detecting and Preventing Deadlocks

12.10.1 Timeouts

Abort a transaction after waiting a certain time for a lock

We won't know if we're in a deadlock, but if a transaction is waiting for way too long without making any progress, abort the transaction.

Good? Simple

Bad? Speculative – wastes work aborting on non-deadlocks

12.10.2 Conservative Two-Phase Locking

Rather than taking the locks as you need them, take them all at once, and release all the locks during commit.

This guarantees no deadlocks, since if you take every lock up-front, then the transaction will succeed.

Bad? Taking a lot of locks upfront and not gradually. Takes more locks and holds them for longer.

12.10.3 Abort On Wait

When a transaction is blocked, abort it or the transaction holding the lock (either).

We are not preventing a deadlock.

Problem? We can potentially abort a lot of transactions that don't need to be (defeatist, wastes a lot of work). At least o deadlocks.

12.10.4 Cycle Detection

A database can maintain a graph on how different transactions are waiting on each other due to locks.

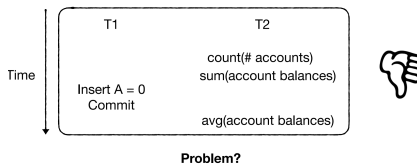
The database builds this graph and detects a cycle. If a cycle is made, it aborts one of the transactions to break the cycle.

Other approaches are speculative. Here, we detect a deadlock for sure. The possible disadvantage? Overhead.

In cycle detection, which transaction should actually be aborted? Ideas? Abort the transaction that:

- Has done the least work
- Furthest from completion
- Been aborted the least times (enforces fairness and avoids starvation)

12.11 Inserts and Statistics (Another Anomaly) – Phantom Reads



The mechanisms we've talked about prior do not solve this.

T1 inserts an account. T2 reports count, sum, and average.

Our existing locks only insert locks on data items as we see them, but this data item might qualify for conditions based on some existing query. So we might end up reading new data where the lock isn't held off.

If these transactions happen serially, it is fine. If it happens in the middle, this could be a problem. T2 only locks existing rows. When a new row is added and committed, there isn't a lock on the new row. This results in inconsistent reporting.

More broadly, this is a **phantom read**: a transaction accesses a set of rows twice, and qualifying rows are added in-between. So we have these anomalies:

- Dirty read (use write locks)
 - Occurs when modified but uncommitted data is read from another transaction
- Unrepeatable read (use read locks)
 - When data is updated in-between while being aggregated multiple times
- Phantom read
 - When data is inserted in-between while being aggregated
 - OR when a data that doesn't meet a WHERE clause was updated to meet the WHERE clause in-between

How do we fix phantom reads?

If we have an index over the table, then there is something interesting you can do. However, in the absence of an index, you can only lock the entire table, and you cannot do anything better than that.

Index locking can address phantom reads. By locking a given node in the B-tree, that allows us to lock a relevant range of keys. For example:

```
SELECT * FROM ACCOUNTS
WHERE ID = ...
```

While doing this, we can take a lock over the corresponding B-tree node for these IDs. These prevent phantom reads, since once we've taken a lock in the B-tree, **you can no longer insert new data items into this particular range**. Hence, you can

prevent qualifying items from being inserted. That is the notion of a ranged lock. Doing this is much better than locking the entire table.

12.12 Transaction Isolation Levels

These allow different types of anomalies to occur. Perhaps your application is ok for these anomalies to occur in exchange for performance. In the table: X means not possible

	Dirty	Un-Repeatable	Phantom
Read uncommitted			
Read committed	X		
Repeatable reads	X	X	
Serializable	X	X	X

What they do:

- Read uncommitted does not hold any locks
- Read committed adds exclusive write locks that are held until transaction end (lock ONLY when writing)
 - This means that shared read locks aren't held, so queries that involve reading could be different when ran more than once
- Repeatable reads hold shared read locks that are held until transaction end
 - At the same time, this also holds exclusive write locks
- Serializable holds range locks until transaction end

What they don't do:

- Read uncommitted: exclusive write locks not held until a transaction end
- Read committed: Shared read locks not held until a transaction end
- Repeatable reads: Range locks not employed
- Serializable: Every enforced

The default is not to be as correct as possible. Some correctness is traded for speed. In many situations, it may be acceptable to give up on phantom reads. If you are running a bank, you probably don't want to run large statistical calculations anyway.

12.13 Impact of Concurrency On Throughput

Ideally, throughput should increase linearly with concurrency. Why does this not happen in practice? There's overhead with locks. It forms an upside-down U curve. Having too many concurrent transactions result in thrashing due to locks. How can we address this?

1. Design the application such that transactions are short
 - a. The shorter the transactions, locks will be held for a shorter duration, so contention between transactions are minimized
2. Restrict the maximum number of concurrent transactions
 - a. You can set this in the DB

As part of a transaction, if there is an index, we want the changes that we make with a transaction to be reflected in an index. It turns out that strict two-phase locking is a potential issue:

- If you lock the root, you'll lock the entire table. **Locking the root is required all the time**
- Lots of locking contention at root and upper level.
- **Index remains locked until the transaction commits**

So effectively, we're taking a read lock over the entire table. We need something a bit more relaxed. Sometimes, we can relax two-phase locking. In this case, this is an example where we need to.

We must maintain a lock on the parent as we acquire a lock on a child, otherwise another transaction may split the child before we get to it.

Double shared locks solve the problem.

Once we actually get to the child, we can release the lock on the parent.

Once we reach the child, we only need to hold a lock on the parent if the child is full, as a split would propagate upwards.

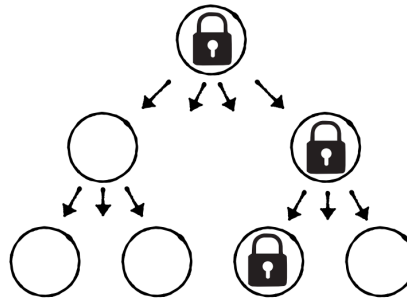
It is ok for a parent to split if the child is locked.

12.14 Lock Coupling / Crabbing

Why is it called that?

It's as if you have a crab that's traversing a tree. It holds two locks at a time when it traverses down the tree.

While strict two-phase locking is correct, it can sometimes be relaxed while maintaining correctness.



13 System Failures

What could cause system failure?

- Media failure (RAID)
- Power failure (logging)
- Data center failure (replication)

Motivating example:

- $A = A - 100$
- Power fails
- $B = B + 100$

We may have persisted this change to the system, but when the system resumes, we see that money has been lost from the system.

What's a high-level solution?

1. Wrap this as a transaction (since the two modifications should be atomic)
2. Record changes to by each transaction in a log, where a log is an append-only file in storage
3. After power fails, scan log and cancel effects of uncommitted transactions

With no "end transaction" flag, undo the operations done. These have performance implications.

13.1 Types of Logging

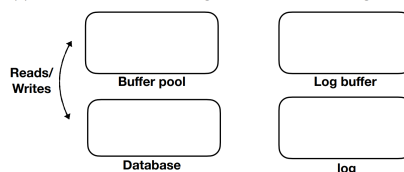
We have undo logging and redo logging. Undo logging results in lots of random I/Os. Redo logging tends to require memory and entails space wastage. It is going to be more constrained with how the buffer pool evicts entries.

We can combine both of these to address both problems.

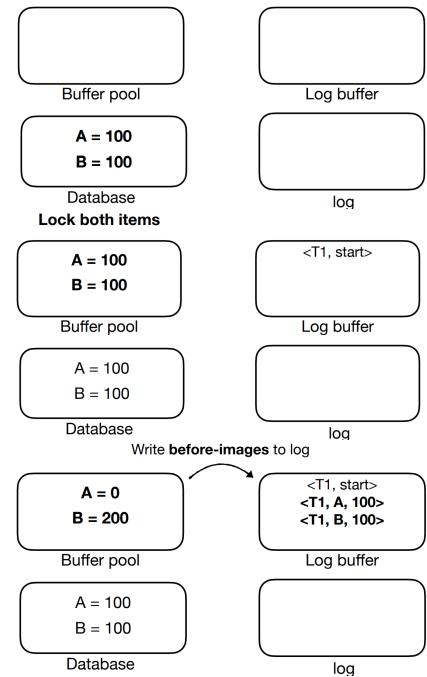
13.2 Undo Logging

Assume relational database.

- (1) write before-image for any changed value to log buffer
- (2) force flush log
- (3) force the changed data into storage
- (4) add commit record to log buffer & force flush again

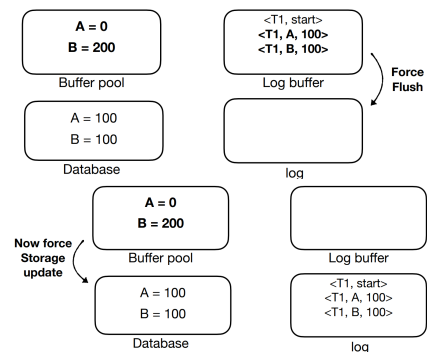


T1: $A = A - 100$, $B = B + 100$

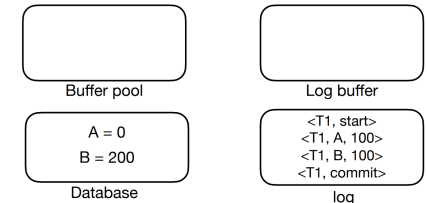


The log will automatically flush if it fills up. However, to **commit** any transaction, we force-flush the log. *A and B cannot be evicted until the log buffer is flushed*

However, to commit any transaction, we force flush the log.



Afterwards, set a commit record <T1, commit> which can be flushed later.



Invariant: if transactions marked as committed in the log, all of its changes are persisted in storage.

Consequence: to recover, undo effects of uncommitted transactions. Identify uncommitted transactions and undo their changes.

Corollary: No constraints on buffer pool eviction policy. The buffer pool is completely autonomous and has no constraints on which data items the buffer pool is allowed to commit or not commit.

How it works, assuming a relational database: If $A = A - 100$ then $B = B + 100$

- Read copies of the data items from

storage into memory (buffer pool). We will now be locking A and B

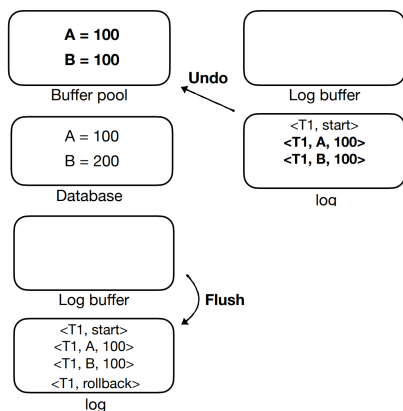
- Store in the log buffer: `<T1, start>`
- Modify both of the data items, in the buffer pool and the log buffer. Log buffer gets `<T1, A, 100>`, `<T1, B, 100>`
- Normally, the log flushes when it reaches capacity. To commit the transaction, a bunch of things have to be done.
 - All the changes made so far must be put into the log in storage, so this forces a buffer flush.
 - Force all the transactions made from the buffer pool to the storage
- To make sure this is commit: Add a commit record to the log's buffer, `<T1, commit>`, and force that into storage. There, the transaction is now committed.

In short:

1. Write before-image for any changed value to log buffer
2. Force flush log
3. Force the changed data into storage
4. Add commit record to log buffer and force flush again

Recovery for undo logging (missing step 4 above): **flush a rollback record**, `<T1, rollback>` so we don't have to undo the transactions again.

Undo any operation of an uncommitted transaction



How much of the log do we actually need to traverse backwards during recovery? **In the worst case, you have to traverse to the start of the log. In other words, the entire log.** It's possible for a transaction that has started and has never finished, where the start was put in front of the log. This results in high recovery time, and we don't want to scan the entire log for recovery.

So, how can we bound recovery time without having to traverse the whole log? **By checkpointing.** In the simplest way of

doing the checkpointing:

1. Stop accepting new transactions and wait until all existing ones commit
2. Flush the log into storage
3. Add checkpoint record to log buffer and flush again

During next recovery, only traverse log up to first checkpoint record we encounter. The checkpoint is going to flush all the transactions from the buffer pool into storage. Once the checkpoint is committed, we put a flag that indicates the checkpoint is committed. Therefore, we only need to scan until the last checkpoint in the log. What are the issues with undo logging? In terms of total work?

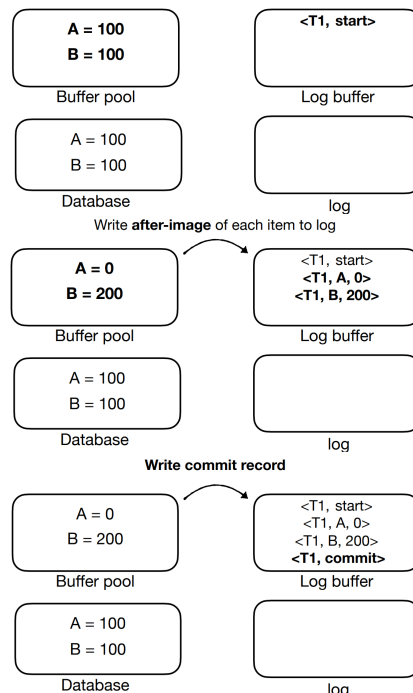
Step 3 is very expensive since it results in many random I/Os (force the changed data into storage). Of course step 2 and 4 are synchronous I/O operations.

If you don't force committed data into storage, when you are recovering, there might be transactions that are committed, but the changes aren't persisted into the database, so you'll have to do a lot of work for committed transactions.

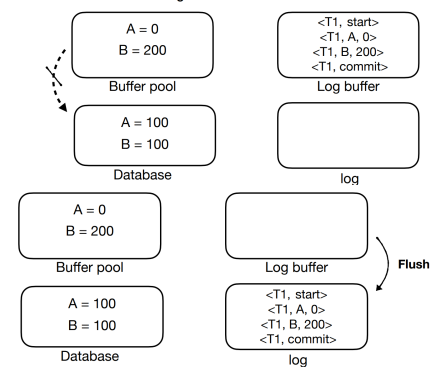
13.3 Redo Logging

Invariant: If transactions is **not** marked as committed in the log in storage, none of its changes have been persisted to the database

Consequences: (1) buffer pool cannot evict uncommitted (dirty) data
(2) while recovering, we can ignore uncommitted transactions
(3) Must redo committed transactions if their changes have not been flushed

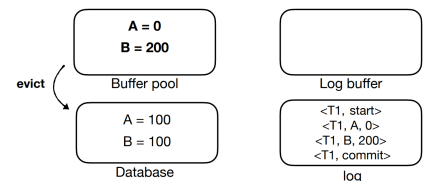


Not allowed to evict until log buffer flushes



Transaction is now committed

Now allowed to evict at leisure

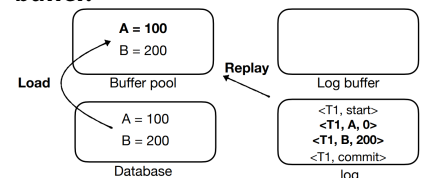


The buffer evicts uncommitted data autonomously based on clock or LRU

Invariant: If transactions are not marked as committed in the log in storage, none of its changes have been persisted to the database.

The buffer pool is not allowed to evict uncommitted or dirty data. If data is dirty, it must stay in the buffer pool until the transaction has been completed.

While recovering, we can ignore uncommitted transactions (because of the invariant). **The difference with re-do logging in comparison to undo logging, is the AFTER-image is stored in the log buffer.**



Once the changes in the log buffer have been evicted into storage, now, the buffer pool is allowed to evict into the database, noting that it doesn't have to do it right away. Maybe a lot more changes can be done to amortize the cost of the I/Os.

To recover, traverse log forward, replaying effects of all the committed transactions. If there is no commit, nothing to do as we know the values have been modified. If committed, we need to read A and B and make a check: are the current values of A and B corresponding to those in the log? Same? Do nothing. If not the same, readjust the values of the data items to what is in the log. The log is the arbitrator of truth. Re-adjusted in the buffer pool. The outline (does not force data into storage):

1. Write after-image for any changed value to the log buffer
2. Keep changed, dirty data in the buffer pool
3. Add commit record to the log buffer and flush
4. Flush changed data to storage at leisure

Note: transactions lock data items, but it can be the case where multiple subsequent transactions can modify the data item. Hence, concurrency is not an issue and the given value is restored to the consistent state if we replay everything.

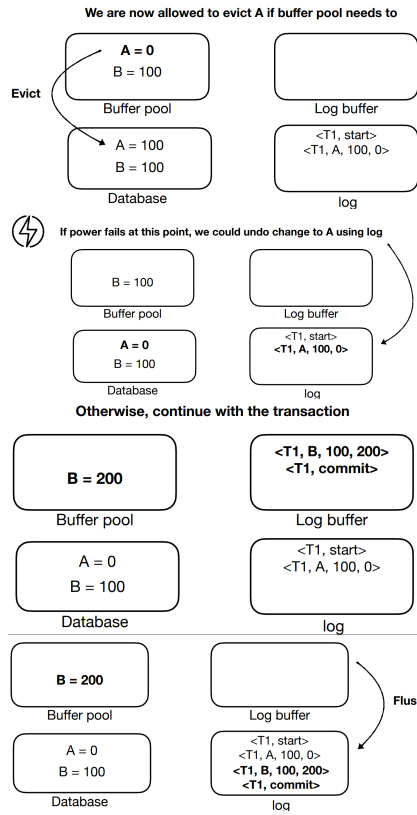
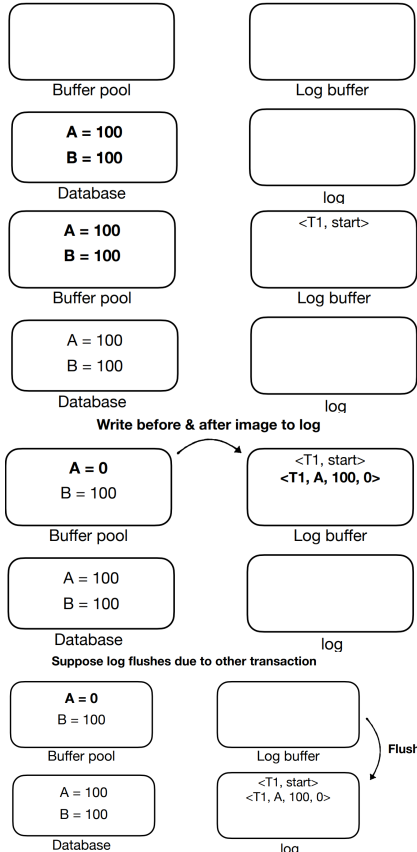
Problem? Here, we don't have any of the I/O overheads that undo logging has.

However, there's going to be a lot of memory overhead since the buffer pool can't evict changed data until the transaction has been committed.

This might also prevent us from evicting the LRU page in the LRU policy, so this might amplify I/O overhead. So, we don't like this.

13.4 Undo/Redo Logging

- (1) write before and after-image for any changed value to log buffer
- (2) before modifying item on disk, must flush log record
- (3) when transaction is finished, flush commit record



We do not need to keep changed data in memory unlike undo logging

Invariant: For any data persisted to storage, its before and after images must first be persisted in the log

Data item granularity rather than transaction-level granularity

Consequences: To recover: **undo uncommitted transactions** and **redo committed transactions**

Corollary: cannot evict dirty data item to storage for which log entry was not already written.

Procedure:

1. Write before and after-image for any changed value to log buffer
2. Before modifying item on disk, must flush log record (full committing is not required)
3. When transaction is finished, flush commit record

Undo-redo logging solves the problems of undo logging resulting in random I/Os and re-do logging holding memory. Still, this type of logging constrains the buffer pool by a bit.

13.5 Checkpointing

1. Stop accepting new transactions and wait until existing ones commit
2. Then flush log to storage
3. Then add checkpoint record to log buffer and flush again

During next recovery, only traverse log up

to first checkpoint record we encounter.

14 Runtime Summary

If internal nodes are not in mem, B-tree Q & IR $\times \log_B(N)$, LSM Q&IR: $\times \log_T \left(\frac{N}{P} \right)$. LSM trees assumed they were clustered. P = no. entries that fit in a buffer

$$L = \log_T \left(\frac{N}{P} \right)$$

\$	Apd. only	Sorted list	B-tree
Q	N/B	$\log \left(\frac{N}{B} \right)$	1
IR	0	N/B	1
IW	$1/B$	N/B	$1 + GC$
S	N/B	$\log \left(\frac{N}{B} \right) + \frac{S}{B}$	$\frac{S}{\text{Clust: } S/B}$
M	B		N/B

\$	LSM-BSC	LSM-TRD	LSM-LVL
Q	$\lg \left(\frac{N}{P} \right)$	LT	L
IR	$\frac{\lg \left(\frac{N}{P} \right)}{B}$	$\frac{L}{B}$	$\frac{LT}{B}$
IW	$\frac{\lg \left(\frac{N}{P} \right)}{B}$	$\frac{L}{B}$	$\frac{LT}{B}$
S	$Q + \frac{S}{B}$	$Q + \frac{S}{B}$	$Q + \frac{S}{B}$
M	N/B	N/B	N/B

\$	LSM-LVL	LSM-BLM	MKY+DV
Q	L	$2^{-M}L$	2^{-M}
IR	$\frac{LT}{B}$	$\leftarrow$	$\frac{L+T}{B}$
IW	$\frac{LT}{B}$	$\leftarrow$	$\frac{L+T}{B}$
S	$Q + \frac{S}{B}$	$Q + \frac{S}{B}$	$Q + \frac{S}{B}$
M	N/B	N/B	N/B

14.1 Bloom Filters

For bloom filters, CPU cost is:

- Insertions = $M \ln(2) = k$, $O(M)$
- Positive query = $M \ln(2)$, $O(M)$
- Avg. Negative query = 2
- FPR = $2^{-M \ln(2)}$

14.2 External Sorting

External sorting:

- $M \geq \sqrt{N \cdot B}$ for one-pass
- If so: $O \left(\frac{N}{B} \right)$ I/O R&W, $O(N \lg(N))$ CPU
- For non-one pass it costs $O \left(\frac{N}{B} \log_M \left(\frac{N}{B} \right) \right)$

14.3 Cuckoo vs. Bloom filter

- FPR:
 - Bloom: $2^{-M \ln(2)}$
 - Cuckoo: 2^{-M+3} , better if $M \geq 10$
- Mem accesses for positive:
 - Bloom: $M \ln(2) = k$

- Cuckoo: 1.5
- Mem accesses for negative
 - Bloom: 2
 - Cuckoo: 2
- Insertion cost:
 - Bloom: $M \ln(2) = 2$
 - Cuckoo: $O(1)$
- Deletes:
 - Bloom: N/A
 - Cuckoo: 1.5
- Storing payloads:
 - Bloom: N/A
 - Cuckoo: Yes

14.4 I/Os For Clustered LSM w/ Monkey vs. Circular

- Updates/deletes:
 - LSM: $O\left(\frac{\log_2\left(\frac{N}{P}\right)}{B}\right) R \& W$
 - Circular: $O\left(1 + 2^{-M+3}\right) R, O\left(\frac{GC}{B}\right) W$
- Inserts:
 - LSM: $O\left(\frac{\log_2\left(\frac{N}{P}\right)}{B}\right) R \& W$
 - Circular: $O\left(2^{-M+3}\right) R, O\left(\frac{GC}{B}\right) W$
 - Add $O\left(\frac{\text{idx size}}{X}\right)$ if we're doing recovery, $O\left(\frac{X}{B}\right)$ for backwards scan is the recovery time, where during recovery **up to** the latest X updates/insertions are scanned
- Gets:
 - LSM: $O\left(1 + 2^{-M \ln(2)}\right)$
 - Circular: $O\left(1 + 2^{-M+3}\right)$
- Scan:
 - LSM: $O\left(\log_2 \frac{N}{P} + \frac{S}{B}\right)$
 - Circular: $O\left(\frac{N}{B}\right)$, not supported
- Memory (**bits per entry**)
 - LSM: $O\left(\frac{K}{B} + M\right)$ where K is the length of the key and B is the no. of keys that fit in a page
 - Circular: $O\left(\log_2\left(\frac{N}{B}\right) + M\right)$. The log term is needed to index the key.

- Recovery
 - LSM: Swift (minimal inconsistent states)
 - Circular: Longer (requires actual thinking)

14.5 SSD WA

SSD write amplification:

SSD WA approximation: where x is % valid pages. WA is at its worst when all erase units have the same number of invalid pages (which means finding the erase unit with the least no. of pages that need to be copied is not going to be so optimal):

$$1 + \left(\frac{x}{1-x}\right)$$

Cost model assuming uniformly randomly distributed writes:

$$1 + \frac{1}{2} \cdot \frac{L/P}{1-L/P}$$

Where:

$$\frac{L}{P} = \frac{\text{valid}}{\text{empty} + \text{valid} + \text{invalid}}$$

14.6 SQL Query Times

Please see section 9

14.7 Column Store Efficiency

Please see section 9

14.8 Log Rules

Log rules:

$$\log_B N = D$$

$$\Rightarrow b^D = N$$

$$\log_a b = \frac{\log_c b}{\log_c a}$$

$$\log_a 1 = 0$$

$$\log_a a = 1$$

14.9 Common Calculations

Common calculations:

$$2^0 = 1$$

$$2^1 = 2$$

$$2^2 = 4$$

$$2^3 = 8$$

$$2^4 = 16$$

$$2^5 = 32$$

$$2^6 = 64$$

$$2^7 = 128$$

$$2^8 = 256$$

$$2^9 = 512$$

$$2^{10} = 1024$$

$$2^{11} = 2048$$

$$2^{12} = 4096$$

$$2^{10} = 1024 = 1\text{KB}$$

$$2^{20} = 1048576 = 1\text{MB}$$

$$2^{30} = 1073741824 = 1\text{GB}$$

$$2^{40} = 1\text{TB}$$

$$1^2 = 2$$

$$10^2 = 100$$

$$100^2 = 10000 = 10\text{K}$$

$$1000^2 = 1000000 = 1\text{M}$$

14.10 Page Read / Write Speed

Accessing sequential pages on the disk is cheap since the seeker does not need to move.

- A disk I/O takes 10ms to read 4KB pages.
 - Throughput: 0.4 MB/s
- An SSD I/O takes 100μs to read 4KB pages.
 - Throughput: 40 MB/s